```
Shell Scripting:
----------------
To run multiple objects[binary/commands] using a single object[file].

.sh || .bash

# vim runme.sh

#!/bin/bash      [env declaration]SHEBANG
cmd1
cmd2
cmd3

:wq [Save & Quit]

# chmod u+x runme.sh
# ./runme.sh
OR
# bash runme.sh



==> Debug:
----------

# bash -x runme.sh
 OR
# vim script.sh
#!/bin/bash
set -x
cmd1
cmd2

# bash script.sh

=> Single line comment
````````````
Add '#'

=> Multiline comments:
''' # start

''' # end

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Variable:
```
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

```

#!/bin/bash
target="example.com"
echo "My target for today is: $target"
echo "My target for today is: ${target}"

#!/bin/bash
read -p "Enter the target: " target
echo "My target for today is: ${target}"
echo "My target for today is: ${target:-DEFAULT}"

#!/bin/bash
target=$(whoami)
echo "My user for today is: $target"

#!/bin/bash
target=`whoami`
echo "My user for today is: $target"

Arithmetic
~~~~~~~~~~~
x=10
y=2
echo $(($x + $y))
----
read -p "Enter first: " x
read -p "Enter second: " y
echo $(($x + $y))

#!/bin/bash
expr 2 + 2
expr 2 - 2
expr 2 / 2
expr 2 * 2
================================
Conditions:
````````````
1) if...then...fi
2) if...then...else...fi
3) if...then...elif...then...fi
4) if...then...elif...then...else...fi









1) if...then...fi
````````````````````
```

```
#!/bin/bash
read -p "Enter: " a

if [[ $a == "0" ]];then
        echo "Ch mayla"
fi
```

2) if...then...else...fi

```
#!/bin/bash
read -p "Enter: " a

if [[ $a == "0" ]];then
        echo "Ch mayla"
else
        echo "Tumse na ho payega"
fi
```

3) if...then...elif...then...fi

```
#!/bin/bash
read -p "Enter: " a

if [[ $a == "0" ]];then
        echo "WoW"
elif [[ $a == "1" ]];then
        echo "Kidding"
fi
```

4) if...then...elif...then...else...fi

```
#!/bin/bash
read -p "Enter: " a
if [[ ${a} == 0 ]];then
        echo "WoW"
elif [[ ${a} == 1 ]];then
        echo "Kidding"
else
        echo "Don't take it lightly"
fi
```

1) Relational Operator:

```
String comparisons: ==, !=, <, <=, >, >=,
Numeric Comparisons: -eq, -ne, -lt, -le, -gt, -ge
RegEx Comparisons:  =~, !~

[[ STR == STR ]]        Equal
[[ STR != STR ]]        Not Equal
[[ NUM -eq NUM ]]               Equal
[[ NUM -ne NUM ]]               Not equal
[[ NUM -lt NUM ]]               Less than
[[ NUM -le NUM ]]               Less than or equal
[[ NUM -gt NUM ]]               Greater than
[[ NUM -ge NUM ]]               Greater than or equal
[[ STRING =~ REGEX ]]   Match RegEx
```

```
[[ STRING !~ REGEX ]]   Not-Match RegEx

if [[ ${var} =~ [0-9a-z] ]];then
        echo "DONE"
fi
```

Password policy rules:

1) Weak passwords

- Are between 1 and 7 characters long (inclusive)
- Contain only letters (a–z, A–Z) and digits (0–9)
- Do not contain any special characters, spaces, or other symbols

2) Normal passwords

- Are between 8 and 12 characters long (inclusive)
- Contain only letters (a–z, A–Z) and digits (0–9)
- Do not contain any special characters, spaces, or other symbols